小项目实施指南：面试弹药库

小项目实施指南：面试弹药库（v3 重写版）

> 2026-10-05 v3 重写（导师与学员讨论定稿）：P1/P2 完成 v3 化之后，小项目的分工也重新做了安排。配套阅读 `03-项目结构定稿.md` （v3） §3。 **定位铁律：小项目不写进简历**，它们的用途只有两个——充当面试弹药，以及给大项目提前扫雷。评价这些项目的标准只有一条：**单位时间内能产出多少面试弹药**。

0. 通用要求

1. 每个项目都要有独立仓库和 README（写清设计取舍、实验数据、复现命令）；代码是写给你自己看的，**真正的弹药是 README 和实验记录**；
2. 每个项目都必须攒齐"弹药三件套"：**一个设计取舍、一个亲自踩过的坑、一张数据或对照表**；
3. **硬性时间盒**：每个项目开工之前，你要在 README 顶部写下截止日期；超期了就砍功能，不能砍质量，更不许对自己说"再打磨一周"；
4. 弹药纪律：项目做完之后立刻写一页《复盘卡》（如果重来会怎么改、被追问时最薄弱的三个点），面试之前你只需要看这张卡；
5. 名额纪律：独立占用工期的小项目**不超过 3 个**——S1 必做，S6（平台岗）与 S4（存储岗）二选一，S3 等你有余力了再做；S2 并入 P1，不单独列出；
6. **微弹药 M1–M3 另成一套体系**（见后文）：它们是按天计的练手小品，算不上项目、不写进简历、不占名额，总投入不超过 1 周。

#	项目	v3 定位	时间盒
S1	mini-Raft KV（Go）	**必做**	3–4 周硬顶
S6	K8s 调度插件实战（Go）	**平台/AI Infra 岗首选，本次新增**	3–4 周（阶段 C 与 P1 步骤 7 合流，净耗时 2–3 周）
S4	bitcask mini 存储引擎（Go）	**存储岗首选，与 S6 二选一**	3–4 周
S3	eBPF 观测工具（Go + bpf2go）	可选，等 P1 核心 1 完工之后再做	1–2 周
S2	C++/io_uring 预取器	**并入 P1**（04 §9.2 加 A 轨实验臂）	随 P1 推进

S1 mini-Raft KV（必做，3–4 周硬顶）

介绍：用 Go 实现一个 3 节点的 Raft，对外暴露 Get/Put 接口。做这个项目的目的，是把一致性从"背出来的知识点"变成"能讲清楚的东西"。 **你要保持清醒**：这是 MIT 6.824 里人手一个的项目，**实现本身的差异化几乎**

为零，简历上永远不要把它排在第一位；它的弹药价值全在工程细节、实验设计和工业代码对读里。

范围：做选举、日志复制、快照、崩溃恢复；**不做**：membership 变更、Pre-Vote（这两个列为"你必须能在白板上讲清楚"的内容）。

关键设计（保留 v2）

- 状态机三个角色，用一个单 `tick` 驱动所有定时器（`time.Timer` 的超时时间在 `[T, 2T)` 内随机化，T 大约取 500ms）；
- **有三个字段必须持久化**：`currentTerm`、`votedFor`、`log[]` ——这是一个经典的死亡陷阱：votedFor 不落盘，节点就可能在同一个任期里投出两票，进而脑裂。WAL 的记录格式是 `len | crc32 | entry`，采用追加写；
- RPC（gRPC）包括 `RequestVote` 和 `AppendEntries`：**收到每个回复都先比 term，发现自己的 term 更小就立刻让位**——这是第二大陷阱；
- 提交规则：leader 只能提交**当前任期**的日志（对应 §5.4.2）；prevLogIndex/term 不匹配时回退 nextIndex，如果你实现了冲突项的加速回退，这是加分项；
- 线性一致读（stretch 目标）：用 ReadIndex 或走日志的方式实现；你要能讲清楚"为什么 ReadIndex 对时钟的依赖比 lease 小"；
- 快照：在 `lastIncludedIndex` 之前截断日志，配合 `InstallSnapshot` 传输；恢复时先装快照，再重放后续日志。

v3 增强（三个动作，把弹药从 60 分提到 85 分）

1. **工业对读（必做）**：toy 版本跑通之后，精读 etcd 的 raft 库——重点看 `node.go` 里的 Ready 循环、`unstable` log、`Storage` 接口，以及 ReadIndex 的工业实现。最后产出一篇《我的 toy raft 与 etcd raft 的 5 个差异》（候选差异：批量化的 Ready 通道、消息流水线、leader 转移、检查点时机、无环日志结构）。"写过一个 lab"和"写过 lab 还读过工业实现"的人，在面试里完全是两个物种；
2. **实验一换血：删掉"30% 丢包单点"这个老实验**（它太极端了，只能得到"它瘫了"这一个结论），改为**丢包率扫描 0.1% / 1% / 5% / 10%**，画出 commit P50/P99 随丢包率变化的曲线，并解释曲线的拐点（重传与选举超时是如何耦合的）；
3. **实验二为新增内容：无主停机窗口的分布**—— `kill -9 leader` 至少 100 次，记录从 kill 发生到新 leader 完成第一个成功写入的时间分布（中位数/P99）。这是一个比提交延迟更受分布式面试官看重的**可用性**数字。分区实验要区分三种：多数派隔离、少数派隔离、不对称分区（leader 单向失联），分别给出行为表。

实施步骤

1. 单机 KV 加 WAL；2. 搭 gRPC 骨架和选举（打印每次 term 变化和投票流向）；3. 复制与提交（把 `raftd` 集成进测试 harness，注入延迟、分区和重启）；4. 快照和 kill -9 恢复；5. 完成 v3 的两组实验，写 etcd 对读笔记。

验收标准：故障注入下，已经提交的数据零丢失；三类分区的行为表完整；丢包扫描曲线画出来；无主窗口分布有至少 100 个样本；etcd 差异笔记写完。 **你能讲的点**：votedFor 为什么必须持久化；为什么只能提交当前

任期的日志；ReadIndex 与 Lease 的对比；**你的 toy 实现与 etcd 的具体差异**；Pre-Vote 如何防止分区节点拿着旧 term 扰乱选举；joint consensus 两阶段成员变更（不要求实现，但要能白板讲）。

S2 C++/io_uring 预取器（v3：不再单列，并入 P1）

定位变化：P1 v3 §9.2 已经把它收编为 A 轨（小文件高 IOPS 臂）的实验分支。你可以为它单独建一个仓库，但它**不占小项目名额，也不另排独立工期**，随 P1 步骤 8 前后推进即可。

范围：做本地写路径和提交模型；**不做**：完整的 S3 客户端（HTTP Range 已经够用）。

关键设计（保留 v2）：

- io_uring（liburing）：队列深度在 64–256 之间可配；用 `IORING_REGISTER_FILES` 加 registered buffer 消掉每个 op 的固定开销；
- **O_DIRECT 三重对齐的坑**：缓冲区地址、长度、偏移都必须对齐逻辑块（用 `posix_memalign` 对齐到 4K）；
- **overlayfs 的坑**：在容器里的 overlayfs 上用 O_DIRECT，可能直接报 EINVAL——目标路径必须落在宿主机真实的 ext4/xfs 上（用 hostPath，或者干脆在宿主机上直接跑）；
- 提交与收割：`get_sqe → prep → submit_and_wait(N)`，CQE 批量收割之后再 resubmit，错误码逐个处理（短读即 EOF）；
- **用 fio 校准**：先测出裸盘直写的上限，然后贴着这个上限比，"我比 fio 差 8%"这种说法比"我很快"专业十倍。

v3 补强：

- 步骤 0 先 `uname -r` 锁定内核版本（registered files/buffers 的行为会随版本变化），并且**把 liburing 官方 examples 逐个通关**——io_uring、O_DIRECT、overlayfs 三个坑叠加在一起，直接上手太陡了，而 C++ 是你的次语言，这个坡度必须削平；
- **收益预期要预先诚实化**：P1 主臂的 4MiB 块经 HTTP 抓取时受网络限速，本地写路径省下的 syscall 对吞吐的贡献预期接近于零；io_uring 真正的战场是 A 轨的小文件高 IOPS。报告里正确的写法是"syscall 数量大幅下降、大块吞吐不变（并解释为什么受网络限速）、小文件臂 IOPS 提升 X"，不许只挑好看的数字报。

实施步骤：1. Go 线程池加 buffered 基线 → 2. C++ buffered 版本（隔离语言因素）→ 3. O_DIRECT 加 io_uring → 4. registered 优化版 → 5. 跑参数矩阵（QD × 块大小 4K/64K/1M × 并发，记录吞吐、strace 统计的 syscall 数、CPU 占用）。 **弹药**：O_DIRECT 对齐与 overlayfs 的坑；io_uring 到底减少了什么（syscall 加线程调度）；**为什么大块场景下它赚不到收益**——能把"不适用的边界"讲清楚，比吹收益值钱得多。

S3 eBPF 观测工具（v3：可选，后置，MVP 砍到只剩一个探针）

定位变化：原来把它排在"P1 指标期"，会和核心 1 的旗舰图争抢时间，现在后移到 **P1 核心 1 完工之后**，有余力了再做；MVP 只做一个探针，不做全家桶。

范围：只做只读观测；要求内核版本不低于 5.10；在宿主机上跑（容器权限很麻烦，先在宿主机上做）。

v3 范围：

- **MVP（唯一必做项）**：block I/O 延迟直方图——用 tracepoint `block/block_rq_issue` 加 `block/block_rq_complete`，按 request 指针配对，在内核侧按 log2 bucket 聚合，用户态周期性拉取 per-CPU map；
- 第二个探针（可选）：major fault（`exceptions/page_fault_user` 里的 major/minor 位）；第三个探针（可选）：`sys_enter_read` 的字节量；
- **有一条观测逻辑必须写进报告**：P1 的 preheat 模型下，缓存命中 page cache 时**根本不会产生 block I/O**——所以 block I/O 直方图天然就能区分命中与未命中；但这里有一个 **readahead 陷阱**：内核预读造成的 major fault 不会记在消费者进程名下，按进程维度归因就会算错。这个陷阱本身就是上等弹药；
- CO-RE：用 vmlinux.h 加 bpf2go；优先用 tracepoint、少用 kprobe（ctx 字段在跨版本时更稳）；准备一个降级方案：采样 /proc/diskstats，并诚实标注两个口径的差异。

实施步骤：1. 跑 bpf2go 的 hello-world 验证环境 → 2. 做 block I/O 配对延迟直方图加 Go 聚合 → 3.（可选）fault/read 探针 → 4. 把 exporter 接入 P1 实验，与 dataloader 的打点交叉验证口径。 **验收标准**：在 P1 实验期间能稳定出数；两个口径交叉验证一致；readahead 归因陷阱有专门一节来讲。 **弹药**：per-CPU map 与 ringbuf 的取舍；为什么 page cache 命中时没有 block I/O；readahead 如何污染进程归因；CO-RE 解决了什么问题。

S4 bitcask mini 存储引擎（存储岗首选，3–4 周）

介绍：用 Go 实现一个 bitcask：WAL 加内存索引，加 hint file，加 merge。 **范围**：只做单机；**不做**：分布式（那部分由 S1 覆盖）、完整的 LSM（与它对读即可）。

关键设计（保留 v2）：

- 记录格式：`crc32 | tstamp | keysz(4) | valsz(4) | key | value`，tombstone 用 valsz 的特殊值标记（删除操作同样是一次写）；
- 内存索引：`map[key] → {fileID, offset, size}`；**1 亿个 key 的内存账，你要在自己机器上实测出数字**（key 平均长度 × key 数量 × Go map 的开销）——这条账决定了 bitcask 的适用边界；
- 活跃文件写满（256MB）之后转为 immutable；**fsync 三档**做成配置：逐条刷盘 / group commit（N 毫秒）/ 只靠操作系统——把三档拿来 benchmark 对比，这是存储引擎最核心的权衡之一；
- merge：把 immutable 文件中的存活记录写入新数据文件和 hint file，然后**原子切换**：先 rename 数据文件和 hint 文件，最后写 manifest 指向新的文件集合；你要把每一个崩溃窗口想清楚，写进注释；
- 读路径：索引命中之后用 pread 读取，再做 crc 校验（校验失败意味着盘上数据腐坏，这时报错或者触发重新 merge）。

v3 的四处补强

1. **时间戳改成 8 字节**：4 字节的秒级时间戳有 2038 问题，记录头统一用 int64 的毫秒或微秒时间戳；主动在注释里写出这个修正——小细节，但能留下大印象；
2. **benchmark 要打到 10M key 这一档**（32G 内存的机器跑得动）：1M 档算不出 bitcask 的命门——全内存索引的容量账，你必须在 10M 档下实测启动时间、常驻内存、读写吞吐，画出"索引内存/数据量"的比值

曲线；

3. **明确实现单写多读模型，并且把它讲透**：写锁天然串行化（这是 bitcask 和 LSM 共同的核心约束），读操作无锁；压测多写者的排队情况和多读的扩展性；
4. **加一次 LSM 对读**（不要求实现）：读 LevelDB/RocksDB 的 compaction 文档，最终目标是能在白板上回答"数据量超过内存之后，bitcask 为什么会输给 LSM（对比 SSTable、稀疏索引、分层 compaction 的读写放大）"。

实施步骤：1. append-only log 加读路径 → 2. 多文件轮转加 tombstone → 3. 启动扫描（先暴力扫，再用 hint file，对比两种方式的启动时间）→ 4. merge 加 manifest 原子切换 → 5. 用 fuzz 随机注入 kill -9，跑完崩溃矩阵 → 6. benchmark：1M/10M 两档、merge 期间的写延迟抖动（**再丑的数字也要敢贴**）、写放大计算、fsync 三档对比。 **验收标准**：崩溃矩阵全绿；10M 档的容量账有实测；merge 抖动数据完整；LSM 对读笔记一页。 **弹药**：fsync 三档；merge 原子切换的崩溃窗口；单写多读约束；10M 实测内存账；bitcask 与 LSM 的适用边界。

S6 K8s 调度插件实战：拓扑感知打分与双目标冲突（新增，平台/AI Infra 岗首选）

介绍：基于 kubernetes-sigs/scheduler-plugins，先跑通官方的拓扑感知调度，再自研一个最小的 PreFilter+Score 插件走通全链路，最后在"数据亲和 × 拓扑亲和"的冲突点上做增量实验。 **为什么选它**：在所有小项目里，它与 P1 咬合得最紧——P1 核心 1 就是 Gate 加 Score plugin，这个项目相当于提前给 P1 步骤 7 扫雷；NUMA 和设备拓扑是 AI Infra 平台岗面试的高频题；scheduler-plugins 是官方仓库，你还可以顺带练习开源流程（**但它不背开源名额**，主要回馈目标仍然是 Fluid/AIBrix）。

事实前提（2026-10 核实，它决定了你的姿势）

- 官方已经提供 `NodeResourceTopologyMatch`（Filter）加 `NodeResourceTopologyResourceAllocationScore`（Score），它们消费 `NodeResourceTopology` CRD（KEP-119），由 NFD（Node Feature Discovery）加 topology-updater 负责上报；
- Topology Manager 从 k8s 1.27 开始 GA；
- **DRA 在今天已经是既成事实，不要再把它当成"未来趋势"来讲**：Dynamic Resource Allocation 从 **Kubernetes v1.35 开始 GA**（stable，特性门控锁定且不可关闭，v1.30 首次引入）；v1.36（2026-05）进一步把 ResourceClaim 接入了新的 Workload/PodGroup（内核 gang scheduling 加 topology-aware scheduling）体系。但 2026 年的现网现实是：NRT 插件仍然是主流，各厂商的 DRA driver 生态还处在迁移期——**所以 S6 正确的叙事是"亲手实现旧设备模型，同时讲清新世界如何取代它"，两边的功夫都要硬**（DRA 白板要点见下）；
- **姿势与大项目同构**：先复现官方实现，再读懂源码，然后写一个最小件走通链路，最后在冲突点上做增量。绝对不要声称"我发明了拓扑调度器"，官方 NRT 插件就是你的对照组。

DRA 白板要点（不写代码，与阶段 A 同期写半页笔记，2026-10 核实）

下面四条你必须能脱稿讲清楚：

1. **旧 device plugin 模型的痛点**：设备被抽象成整数资源（比如 `nvidia.com/gpu: 1`），你无法按显存、型号或互联拓扑发出结构化请求，它不支持设备共享，也不支持按属性表达式过滤，拓扑信息只能走 Topology Manager + NFD + NRT 这条外挂链路；
2. **DRA 的四个核心对象**（`resource.k8s.io/v1`）：驱动用 **ResourceSlice** 上报设备能力与拓扑，集群管理员定义 **DeviceClass**（可以类比 StorageClass），工作负载用 **ResourceClaim / ResourceClaimTemplate** 申领设备（体验类似 PVC/PV 的申领），调度器据此做结构化匹配，kubelet 与驱动之间完成分配握手；它支持 CEL 属性过滤和设备共享；
3. **它与 S6/NRT 的关系**：DRA 让设备拓扑成为调度器可以直接消费的结构化属性，长期来看会收编 NFD+NRT 这条外挂链路的一部分；但迁移期两套机制并存，NRT 插件在现网中仍被广泛部署——你能讲清"哪部分会被取代、哪部分（NUMA/CPU 亲和）仍会独立存在、迁移卡在哪里（驱动成熟度与存量集群版本）"，才是满分答案；
4. **前沿联动**：v1.36 的 Workload/PodGroup API 在内核里做 gang scheduling 加 topology-aware scheduling，并且接入了 DRA——这与 P1 的训练 Job 调度、S6 的拓扑亲和是同一个演进方向，面试时可以当作"我在跟踪内核调度演进"的谈资，但明确不跟进实现。

两小时手感练习（可选，不过夜、不写简历、不挤占 P1）：阶段 A 完成之后，用 kind 建一个 1.35 以上的集群，跑官方的 `dra-example-driver`（一个不需要真实硬件的假设备驱动），创建一个 DeviceClass 加一个 ResourceClaim，`kubectl describe resourceclaim` 能看到分配结果和 ResourceSlice 的内容就收工。这个练习的目标只是让你"见过真正的 ResourceSlice/ResourceClaim 长什么样"。

硬件诚实清单（开工第一件事）

- 先跑一次 `numactl -H` 确认：游戏本的消费级 CPU 几乎肯定是**单 NUMA 节点**，也没有多 GPU；
- 所以所有拓扑都用 fake/annotation 驱动来开发——**这完全正当**，官方插件自己的 e2e 测试也在 mock NRT 对象；
- 真机多 GPU 拓扑只在 P2 租云主机拿真值的那个窗口顺带读一次（`nvidia-smi topo -m`），不值得为它单独开预算。

阶段 A：现勘并跑通官方链路（约 1 周）

1. 搭好 kind 集群（版本对齐 scheduler-plugins 的兼容矩阵），按仓库 Developer Guide 在本地构建镜像（kind load）或者直接用发布的 chart；
2. 部署 NFD 加 topology-updater，生成真实的 `NodeResourceTopology` CR；在单 NUMA 机器上手写一份 fake NRT yaml（构造双 NUMA、fake GPU 资源池），作为第二个数据源；
3. 配置 **v1 版本**的 `KubeSchedulerConfiguration`：包括 `profiles`（schedulerName）、`plugins`（在 filter/score 上启用 NRT 插件）、`pluginConfig`（scoringStrategy：MostAllocated、BalancedAllocation、LeastAllocated 各跑一遍，观察决策差异）；
4. 部署若干请求 CPU/GPU 的测试 Pod，验证绑核与拓扑决策是否符合预期，抓取调度日志；
5. 精读：官方 Score 插件源码（分数如何由 NRT 的 zones 和资源分配算出、如何归一化）、NRT CRD 的字段语义；
6. **验收标准（你能口述全链路）**：device plugin 的 `TopologyInfo` → kubelet Topology Manager（与 CPU Manager 协同）→ NFD topology-updater → NRT CRD → 调度插件 Filter/Score → bind。

7. **风险保护**：如果一周之内官方全家桶部署不成功（NFD 组件多、镜像拉取坑也多），S6 立即降级为"手写 NRT yaml + 只部署插件本体"；再不行就降级为"纯跟读源码 + 写决策日志推演笔记"，不能让环境黑洞吃掉第二周。

阶段 B：自研最小插件（1–2 周）

1. 插件骨架：实现 `New(obj runtime.Object, handle framework.Handle) (framework.Plugin, error)`，挂上 **PreFilter 与 Score 两个扩展点**（NormalizeScore 视需要再加）；
2. 配置：在 `New` 里读取 `pluginConfig` 参数（权重、annotation key），保存到插件状态中；
3. **拓扑数据源刻意做简化**：先读 node annotation（比如 `topology.io/numa-zones: ...`，里面写 fake 拓扑 JSON），不依赖 NFD 全家桶——这一步的目标是走通链路，我们不重造上报机制；
4. 打分逻辑：根据 Pod 请求的资源与节点的 zone 拓扑，计算"单个 NUMA 内的可满足度"分数；PreFilter 先排除明显不可行的节点；
5. 工程配套要做全：自定义 scheduler 的 Deployment、ServiceAccount/RBAC（这份调度器权限清单本身就是弹药）、用 ConfigMap 挂载 KubeSchedulerConfiguration；
6. Profile 配置：用 YAML 定义 profile 与插件的启用顺序；本地调试按 Developer Guide 来（本地跑 scheduler 进程连 kind apiserver，或者容器化调试，二选一）；
7. 质量：插件单测（构造 nodes/pods 的 table tests）+ e2e（调度结果符合 annotation 的预期）+ 每个调度决策都打一条结构化日志。
8. **验收标准**：在一个独立的 `schedulerName` 上，"配置 → 编译 → 部署 → 调度生效"全链路走通，单测和 e2e 全绿。

阶段 C：双目标冲突实验（1–2 周，与 P1 步骤 7 合流，不额外占工期）

1. 让两个 Score 维度并存：**数据亲和分**（P1 的 Job 子集 readyRatio，由 P1 的 informer 缓存提供）× **拓扑亲和分**（fake NUMA/GPU zone 的匹配度）；
2. 构造冲突场景：Job 的热分片缓存在 NUMA-A 节点上，但空闲 GPU 在 NUMA-A'（拓扑上靠近远端内存）；或者另一个节点 NUMA 局部性很好，但数据是冷的；
3. 权重 `w` 从 0 扫到 1，画出**权重-调度决策曲线**，以及对应的 makespan/拓扑惩罚代理指标；
4. 把终点问题写成报告：框架级的多插件分数如何归一化（各插件 Score 的值域）、冲突时为什么不存在银弹、生产环境中应该让谁把权重暴露给用户；
5. **合流动作**：这个插件的 Score 实现直接演化为 P1 步骤 7 调度插件的骨架，S6 仓库里保留独立的 demo 版，P1 里用生产版；
6. **验收标准**：产出一张冲突权重-决策曲线图；DRA 四条白板要点能脱稿讲（见上，包含 v1.35 已 GA 的现状判断，禁止再说"DRA 是未来的实验特性"）。

弹药三件套：取舍（NRT annotation fake 与 NFD 全家桶之间怎么选、双目标如何归一化）、坑（KubeSchedulerConfiguration 的 apiVersion、多 profile、schedulerName、RBAC 权限、kind 镜像加载）、数据（权重-决策曲线加 scoringStrategy 对照）。 **诚实清单**：拓扑全部是 fake 的；官方 NRT 插件是基线，不是要打倒的对手；没有做绑核验证，也没有真机性能数据；DRA 没有动手实现，但它的事实状态（v1.35 GA）和四个要点你必须跟到能讲、能回答迁移期共存问题的程度。

微弹药 M1–M3（按天计的练手小品，算不上项目）

定位纪律（违反了它们就失去意义）：

1. **它们不是项目**：一个目录加一份 `NOTES.md` 就够了，不做 README 包装、不写进简历、不占小项目名额——面试官见多了"手写 Docker"这种水分项目，但"我写过、代码就在这里、原理我讲得清楚"就是真弹药；
2. **硬性时间盒**：单个不超过 2–3 天，三个全做的净工作量不超过 1 周；一旦超时，立即走该条目的降级路径，不过夜、不打磨；
3. **窗口纪律**：只允许在寒假、实习空档或两个大项目的里程碑之间做；P1 核心 1 冲刺期（步骤 7 前后）禁止开工；
4. **顺序**：M1 所有人必做；M2/M3 按岗位选——走平台/调度方向优先做 M2，走存储/数据流方向（你的 P1）优先做 M3；有余力再全部做完。

M1 mini-container（必做，1–2 天）⭐ 性价比最高

做什么：用 Go（或者直接用 C 调 clone）写一个 100–200 行的容器运行器。

- 用 `clone` 加 `CLONE_NEWPID/NEWUTS/NEWNS`（有余力再加 NEWNET/NEWUSER）；
- cgroup 用 **v2**：在 `cgroup.controllers` 体系下写 `cpu.max`、`memory.max`、`memory.swap.max`，把子进程的 pid 写入 `cgroup.procs`；
- rootfs 用 busybox 或 distroless 的根目录，调用 `pivot_root`（入门版可以先用 chroot）切换，再挂载 `/proc`、`/tmp`；
- stretch（可选）：用 overlayfs 的 lower/upper/merged 做一个可写层——这正好与 P1 里"overlayfs 上 O_DIRECT 的坑"呼应。

验收标准：容器内执行 `ps` 只能看到自己的进程树；hostname 相互独立；把 `memory.max` 设成 64M 后跑内存分配，能触发 cgroup OOM 并观察宿主行为；用 `lsns` 能看到新的命名空间。 **你必须能讲**：容器为什么不是虚拟机（共享宿主内核意味着什么、攻击面在哪里）；cgroup v2 相对 v1 的统一层级变化；pivot_root 与 chroot 的区别；为什么 user namespace 是 rootless 容器的关键；containerd/runC 在 K8s 链路中的位置（kubelet → CRI → containerd → runC）。 **降级路径**：pivot_root 卡住就退回 chroot 加 mount namespace，核心目标（找到 namespace/cgroup 的手感）不打折。

M2 mini-CNI（平台/网络向，1–2 天）

做什么：写一个能被 CNI 规范真正调用起来的最简网络插件（Go 可执行文件）。

- 实现 CNI spec 的 `ADD` / `DEL` / `CHECK` 三个命令：从 stdin 读 NetConf JSON，从 `CNI_ARGS` 和环境变量里拿 netns 路径；
- bridge 模式：建好 bridge，给容器 netns 创建 veth pair 并把一端塞进去，从 /24 地址池（用位图或租约文件持久化，防止 ADD 重启之后地址泄漏）分配 IP，配好接口地址与默认路由，再把 lo 拉起来；
- 用 `cnitool` 在两个独立的 netns 上验证互通；DEL 时释放 IP、删掉 veth。

验收标准：两个 netns 之间能 ping 通；重复 ADD 不会撞 IP；DEL 之后地址正确回收；用 `tcpdump` 能抓到 bridge 转发的包。 **你必须能讲**：K8s 的"IP-per-pod"网络模型；CNI 在容器创建链路中是哪个时刻被调用的

（netns 已经建好、rootfs 已经就位之后，由运行时/CRI 调用）；跨节点流量怎么办（VXLAN、路由、BGP——这个练习故意不做，但你要能说清边界在哪里）；service 的 iptables/IPVS 与 CNI 为什么是两层东西。 **降级路径**：如果 CNI 调试吃掉了半天还没结果，就停下来，改为"读 containernetworking/plugins 的 bridge 插件源码，再用 `ip link/addr/route` 手敲复现全链路"，然后写笔记。

M3 mini-CSI（存储/数据流向，2–3 天）⭐ 与 P1 咬合最紧

做什么：写一个最小的 CSI 驱动，让它真的被 kubelet 调起来，让 Pod 能挂载你提供的卷。

- 实现三个 gRPC service：`Identity`（GetPluginInfo/Capabilities/Probe）、`Controller`（最小实现，先不做 provisioning）、`Node`（在 `NodePublishVolume` 里把宿主目录 bind mount 到 kubelet 给的 target path；在 `NodeUnpublishVolume` 里反向卸载）；
- 走 **kubelet 的 plugin registration 机制**：在 `/var/lib/kubelet/plugins_registry/` 暴露注册 socket，与 kubelet 完成握手（这一步本身就是最有价值的弹药）；
- 先打通 **static PV 加你的 CSI driver** 的绑定挂载闭环；stretch 目标：实现 `CreateVolume/DeleteVolume`（在宿主上建/删目录）加 StorageClass 动态供给；
- 在 kind 里可以用 DaemonSet（privileged、挂 hostPath）部署，也可以直接作为节点进程跑，两种形态里选简单的一种先跑通。

验收标准：PVC 完成绑定 → 测试 Pod 挂载成功并能读写文件 → 删掉 Pod/PVC 之后 bind mount 被干净卸载（用 `findmnt` 验证没有残留）。 **你必须能讲**：CSI 三段式流程——Attach（ControllerPublish）、Stage（NodeStageVolume，块设备格式化并挂到全局挂载点）、Publish（bind mount 到 Pod）各自解决什么问题，为什么你的最简实现只需要 Publish；kubelet-registration 握手与 socket 的生命周期；ephemeral 卷与 persistent 卷的区别；**Fluid/JuiceFS 为什么需要自己的 CSI driver**（FUSE 挂载如何经由 CSI 注入 Pod）——最后这一问直接和 P1 现勘连起来。 **降级路径**：动态供给做不完就停在 static PV 闭环（80% 的弹药已经到手）；在 kind 里注册不上，就在节点上用 `csc` 工具直接调你的 gRPC 方法来验证。

微弹药与面试域的关系

微弹药	覆盖的面试域	联动
M1	容器运行时、namespace/cgroup、CRI 链路	S6（K8s 的下层）、S2 的 overlayfs 坑
M2	CNI、容器网络模型、netns	S6 调度之后"Pod 如何落地并联网"
M3	CSI、kubelet 卷生命周期、存储编排	**P1 现勘（Fluid/JuiceFS CSI）**、S4 存储岗叙事

完成顺序与依赖（v3 日历）

```
现在（使用阶段）
  ├─ S1 起步（穿插进行，3–4 周硬顶；在 P1 早期完成）
  └─ P1/P2 双现勘（主战场）
P1 主线期
  ├─ S6 阶段 A/B（用周末预算，约 2–3 周）
  ├─ P1 步骤 7 ← S6 阶段 C 合流（不另占工期）
  └─ S2 随 P1 A 轨推进（不单列）
实习投递（2027.2–4）：P1 核心 1 + S1 + S6（平台岗）/ S4（存储岗）+ Fluid issue
P1 里程碑之间 / 寒假：M1 必做（1–2 天）；M3 优先（存储/P1 向，2–3 天），M2 面向平台岗
暑期实习后 / 秋招前：S3 用来填空档（前提是 P1 核心 1 已经完工）；如果改投存储岗，S4 放在这个窗口做
```

选岗规则：主攻 AI Infra 平台/调度方向，就选 S6 加 M2；主攻分布式存储/数据流方向，就选 S4 加 M3。不要贪心，正式小项目二选一；M1 无条件做，M2/M3 按岗位选，微弹药总耗时全部加起来不超过 1 周。